

AI Laboratory — Bayesian Networks and Autoregressive Language Models

First-Order and Second-Order Probabilistic Language Models

1. Part I — From Probability to Language

Question 1: The chain-rule decomposition is useful for generation because it expresses a complete sequence probability as a product of next-token conditional probabilities. This lets generation proceed one token at a time: sample the first token, condition the next token on the available history, and repeat. A simple autoregressive model approximates the full history with a restricted context.

$$P(x_1, \dots, x_t) = P(x_1) \times P(x_2 | x_1) \times \dots \times P(x_t | x_1, \dots, x_{t-1}).$$

2. Part II — Bayesian Network for Text

Question 2: The first-order network assumes that the current token is conditionally independent of all earlier tokens once the immediately preceding token is known:

$$P(X_t | X_1, \dots, X_{t-1}) = P(X_t | X_{t-1}).$$

This gives the first-order Markov factorisation $P(X_1, \dots, X_t) = P(X_1) \prod_{t=2}^t P(X_t | X_{t-1})$.

3. Part III — Dataset

- the cat sat on the mat
- the cat sat on the rug
- the dog sat on the mat
- the dog ran to the park
- the cat ran to the park
- the dog sat on the rug

The sentences were lower-cased, split into word tokens, and augmented with <START> and <END>.

4. Part IV — First-Order Conditional Probability Table

Question 3: The first-order CPT is constructed by counting each observed consecutive-token transition and normalising each row by its total outgoing transition count.

Current token	P(next word current token)
the	cat: 0.250, dog: 0.250, mat: 0.167, park: 0.167, rug: 0.167
cat	ran: 0.333, sat: 0.667
dog	ran: 0.333, sat: 0.667
sat	on: 1.000
ran	to: 1.000

Unobserved transitions have probability zero in this unsmoothed model. For example, $P(\text{rug}|\text{cat})=0$ and $P(\text{park}|\text{sat})=0$.

Dataset note: the laboratory gives a separate hypothetical illustration in which “the cat” occurs three times and “the dog” twice, giving $3/5$ and $2/5$. In the actual six-sentence dataset, “the” has 12 outgoing transitions: cat=3, dog=3, mat=2, rug=2, park=2. Thus the actual supplied-data values are $P(\text{cat}|\text{the})=3/12=0.25$ and $P(\text{dog}|\text{the})=3/12=0.25$. The $3/5$ and $2/5$ values are therefore treated as the handout's illustrative hypothetical example, not as the result for the supplied corpus.

5. Part V–VI — LLM Implementation and Inspection

Prompt used:

Write a simple Python implementation of a first-order autoregressive language model.

The model should:

- 1. take a list of tokenised sentences as training data;*
- 2. count transitions between consecutive tokens;*
- 3. construct the conditional distribution $P(X_t | X_{t-1})$;*
- 4. display the probabilities for a specified previous token;*
- 5. predict the most probable next token;*
- 6. generate a sentence by repeatedly sampling the next token;*
- 7. stop when the <END> token is generated.*

Do not use a machine-learning library or a pretrained language model. Use ordinary Python data structures and random sampling.

Question 4: Transition counts are stored in a nested mapping/Counter keyed by the previous token and then by observed next token.

Question 5: Each conditional probability is computed by dividing a transition count by the total number of transitions observed from the corresponding previous-token context.

Question 6: Greedy prediction selects the arg-max next token, whereas sampling draws according to the full probability distribution. Greedy generation is deterministic for a fixed model and start state; sampling allows variation.

Question 7: If a context has no observed transition, no probability row exists. The implementation returns None and generation stops safely rather than inventing an unseen transition.

6. Part VII — Probability Normalisation

Question 8: A conditional row summing to 0.87 indicates that the probabilities are not correctly normalised. Some probability mass is missing or the calculation is wrong. Since rows of a conditional distribution must sum to 1, this is a direct implementation error to investigate.

All observed first-order and second-order rows in the implementation sum to 1 within numerical tolerance.

7. Part VIII — Next-Word Prediction

Preceding token	Most probable next token
<START>	the
the	cat
cat	sat
dog	sat
sat	on
ran	to
on	the

Question 9: No. The most probable next token is determined by the observed frequencies in the tiny training corpus. Human expectations can draw on semantic, syntactic, and world knowledge that this first-order model does not encode.

8. Part IX–X — Generate Text

Twenty sampled first-order sentences (random seed 42):

1. the cat ran to the park
2. the dog ran to the cat ran to the cat sat on the park
3. the cat sat on the cat sat on the park
4. the dog sat on the rug
5. the cat ran to the cat ran to the dog sat on the rug
6. the cat sat on the rug
7. the park
8. the cat ran to the cat sat on the mat
9. the dog ran to the dog sat on the cat sat on the cat ran to the dog ran to the mat
10. the cat sat on the dog sat on the dog sat on the mat
11. the rug
12. the mat
13. the mat
14. the cat ran to the rug
15. the cat sat on the dog ran to the cat sat on the rug
16. the mat
17. the dog sat on the mat
18. the dog sat on the cat sat on the rug
19. the park

20. the rug

Five greedy first-order generations:

1. the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat
2. the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat
3. the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat
4. the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat
5. the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on the cat

Question 10: Sampling produces more variation because it can select different next tokens according to their probabilities. Greedy generation always selects the highest-probability transition for a context, so it is deterministic for the same starting state and model.

9. Part XI–XII — Second-Order Bayesian Network

Question 11: The second-order model estimates $P(X_t|X_{t-2}, X_{t-1})$ and has the structure $X_{t-2} \rightarrow X_t \leftarrow X_{t-1}$.

Graph structure: First-order uses one preceding token as context; second-order uses two preceding tokens.

Conditional probability table: First-order rows are indexed by individual tokens; second-order rows are indexed by ordered token pairs.

Context available: Second-order prediction has one additional token of history.

Amount of data needed: More data is required because there are many more possible contexts and each context may occur less often.

Second-order context	P(next context)
(<START>, the)	cat: 0.500, dog: 0.500
(the, cat)	ran: 0.333, sat: 0.667
(the, dog)	ran: 0.333, sat: 0.667
(cat, sat)	on: 1.000
(dog, sat)	on: 1.000
(dog, ran)	to: 1.000
(sat, on)	the: 1.000

10. Part XIII — Comparing the Two Models

Non-zero first-order parameters: 17

Non-zero second-order parameters: 19

Observed first-order contexts: 11

Observed second-order contexts: 15

Vocabulary size including <START>/<END>: 12

Unobserved possible single-token context rows: 1

Unobserved possible ordered-pair context rows: 129

Five second-order greedy generations:

1. the cat sat on the mat
2. the cat sat on the mat
3. the cat sat on the mat
4. the cat sat on the mat
5. the cat sat on the mat

Five second-order sampled generations:

1. the cat ran to the park
2. the cat ran to the park
3. the dog ran to the park
4. the cat ran to the park
5. the dog sat on the rug

Question 12: Increasing context can improve prediction because the model can distinguish more detailed histories. However, the conditional table grows rapidly and a small dataset supplies fewer observations for each specific context. This creates sparsity and makes reliable estimation harder. The trade-off is richer contextual information versus greater data requirements.

11. Part XIV — Connection to Modern Language Models

Modern autoregressive language models retain the same probabilistic objective of estimating the next-token distribution conditioned on previous tokens. The simple laboratory models use explicit count-based CPTs; modern neural language models use neural networks to estimate the conditional distribution.

Simple BN / n-gram	Modern autoregressive LM	Shared idea
CPTs	Neural network	Conditional next-token distribution
Fixed context window	Large learned context	Predict token from previous tokens
Explicit probabilities	Learned weights	Estimate conditional probabilities
Counting	Gradient-based training	Learn from data
Sampling	Sampling	Generate token by token

12. Part XV — Reflection on the Role of the LLM

Question 13: Approach B is preferable because it specifies the intended probabilistic behaviour before code is generated. The representation, conditional distribution, training signal, and generation mechanism are explicit, which makes the implementation inspectable and testable. It supports direct checks such as probability normalisation and verification of known transition probabilities. A vague request leaves the model representation

and behaviour unspecified, making it harder to determine whether the generated code implements the intended intelligent system.

The LLM was used as an implementation assistant. The engineer remained responsible for understanding the probabilistic model, inspecting the generated representation, executing the code, and validating probabilistic invariants.

Example of inspected implementation: transition counts are kept separately from the normalised probability table, and the row total is explicitly used to calculate each conditional probability. The generation code also separates arg-max prediction from random sampling so that the requested generation modes are not conflated.

13. Final Question — What Did the Bayesian Network Add?

Question 14: Viewing the language model as a Bayesian network makes token dependencies explicit, provides a factorisation of the joint distribution, gives a direct interpretation to conditional probabilities, and provides a principled token-by-token generation procedure. It also exposes the independence assumption of a first-order model and clarifies how increasing context changes the graph and the CPT. Finally, the probabilistic representation gives testable invariants—such as each conditional row summing to one—that help determine whether the implementation matches its specification.

14. Validation Summary

Check	Result	Observation
First-order normalisation	PASS	Every observed row sums to 1.
Second-order normalisation	PASS	Every observed row sums to 1.
P(cat the) on supplied dataset	PASS	$3/12 = 0.25$.
P(dog the) on supplied dataset	PASS	$3/12 = 0.25$.
Twenty sampled sentences	PASS	20 sentences generated.
Greedy vs sampling	PASS	Both generation modes implemented and compared.
Second-order generation	PASS	Greedy and sampling modes implemented.